

Algorithm (Stage 1)

1. Input & Pre-Processing

- i. **Courses (C):** Set of all courses to schedule, e.g., $C = \{c_1, c_2, \dots, c_n\}$. Each course c_i has an attribute: a set of suitable rooms the course can be scheduled [a subset of the Rooms set in (v)].
- ii. **Sections (S):** A course CSC310 may be done by four programs: Computer Science (CSC), Data Science (DTS), Management Information Systems (MIS), and Fin. Tech. (FTH). If there are two divisions of the course, such that CSC and FTH constitute one section, and DTS and MIS constitute another section. This means that CSC310 **would have two sections**, and that will influence the schedule. **A section is a derivative of a course**. The set of all sections $S = \{s_1, s_2, \dots, s_n\}$. Each s_i has attributes: enrollment size (enroll_si), a set of lecturers (S_i^l), and student groups (S_i^g) taking the course.
- iii. **Events (E):** Divide the sections into events, such that a course that requires more than two hours will be divided into multiple sections. For example, a three-unit course will be divided into two events (one for 2 hours and the other for an hour). **An event is a derivative of a section**. Let $E = \{e_1, e_2, \dots, e_n\}$ represent a set of all events.
- iv. **Timeslots (T):** Set of available timeslots per day, e.g., $T = \{t_1, t_2, \dots, t_m\}$ (discrete slots like "Monday 9-11 AM"). Timeslots should be implemented as a class because of the different university structures.
- v. **Rooms (R):** Set of available rooms, e.g., $R = \{r_1, r_2, \dots, r_k\}$. Each r_j has attributes: capacity (cap_rj), features (features_rj), availability per timeslot (avail_rj_t, a boolean matrix).

2. Conflict Graph Construction

- i. Vertices $V = C$ (one per course). This means that if there are
- ii. Edges E : Create an edge between e_i and e_j if they cannot be assigned the same timeslot. Reasons for conflict:
 - a. Same lecturer: $\text{lect_}e_i == \text{lect_}e_j$.
 - b. Overlapping students: $\text{students_}e_i \cap \text{students_}e_j \neq \emptyset$.

- c. Other hard rules (e.g., departmental/university policies requiring separation).
- iii. This graph captures the "colouring" aspect: Adjacent events must get different "colours" (timeslots).

3. CSP Model Components

The CSP is defined as a triple $(X, D, \text{Constraints})$, where:

Variables (X): One variable X_i for each event $e_i \in E$. X_i represents the assignment of a timeslot and room to e_i .

Domains (D): For each variable X_i , the domain D_i is the set of possible assignments (t, r) where:

$t \in T$ (a valid timeslot).

$r \in R$ (a valid room).

Constraints on suitability:

- i. Room is available at t .
- ii. Room capacity sufficient.

D_i **must be** precomputed for each e_i to reduce the search space.

Constraints:

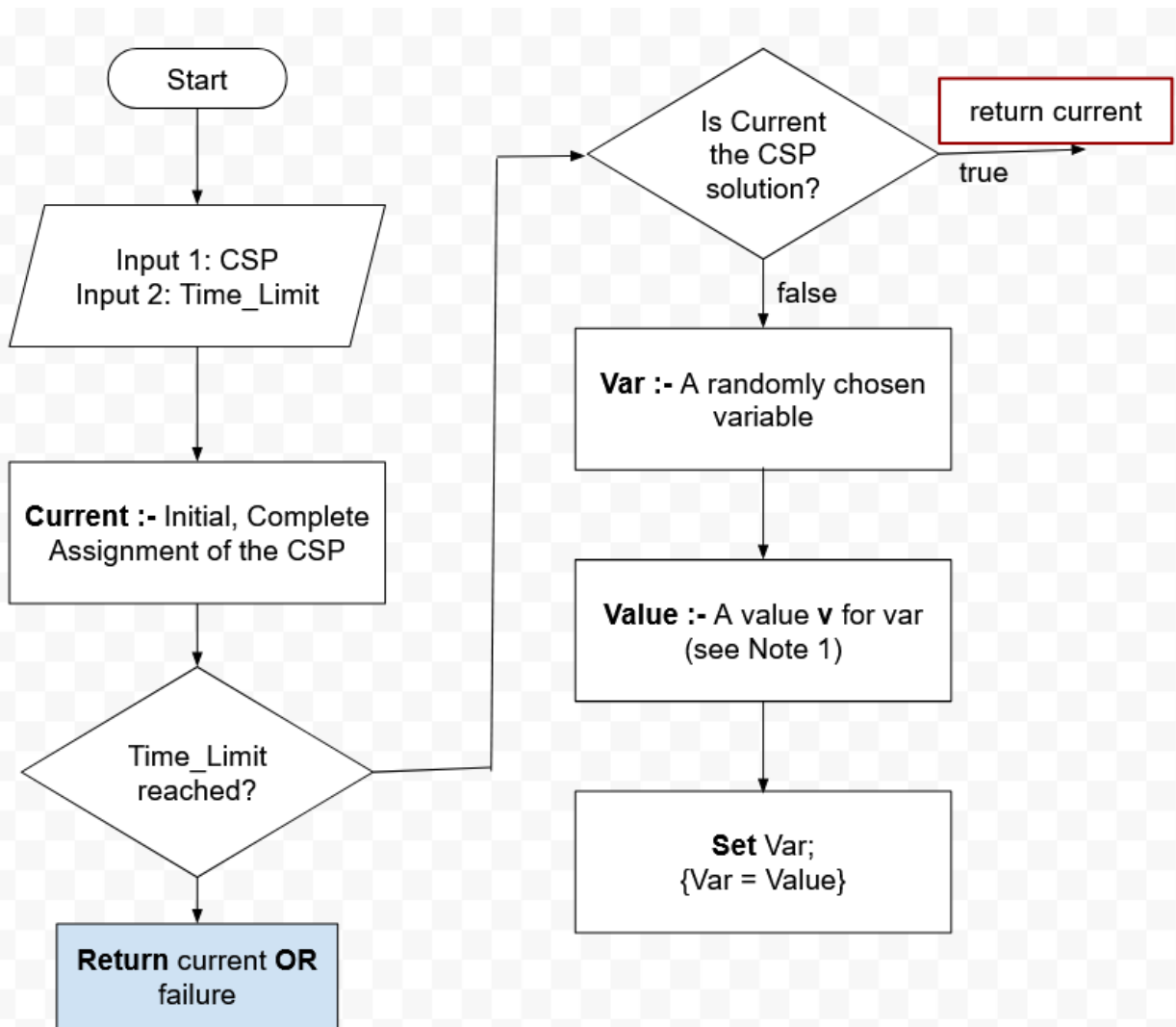
- i. **Unary Constraints** (on single variables, enforced during domain preprocessing):
- ii. **Binary Constraints (on pairs of variables):** Colouring Constraint (No Time Conflict): For every edge $(e_i, e_j) \in E$, the timeslots must differ: if $X_i = (t_i, r_i)$ and $X_j = (t_j, r_j)$, then $t_i \neq t_j$.
- iii. **Room Exclusivity Constraint** (No Room Overlap at Same Time): For every pair of events e_i, e_j ($i \neq j$), if $t_i = t_j$, then $r_i \neq r_j$. (This ensures no two courses use the same room at the same time, regardless of conflicts.)
- iv. **Global Constraints** (optional for efficiency, but can be modelled as multiple binaries): All rooms are uniquely assigned per timeslot (equivalent to AllDifferent on rooms for courses in the same t).

These constraints ensure the solution satisfies all hard rules: no student/lecturer clashes (via colouring), no room double-booking, and suitability.

This formulation turns UCTP into a graph colouring problem within CSP: The colouring enforces different timeslots for conflicting courses, while additional constraints handle rooms like a matching or assignment problem per timeslot.

Method 1: Local Search for solving the formulated CSP

In the flowchart below, we describe one of the fastest methods to solve a huge CSP as this.



Note 1: the value v for var that minimises $CONFLICTS(csp, var, v, current)$, where $CONFLICTS(csp, var, v, current)$ returns the number of conflicts that would occur if v is assigned to var within $current$ (i.e. the current schedule).